

Guia de Estudo: Desenvolvimento de Sistemas Robustos com Python, POO e Flask

Este Guia de Estudo serve para indicar o conteúdo apresentado durante as aulas de forma resumida e dinâmica. Para um melhor aproveitamento, pesquise o conteúdo completo nas aulas. Foi utilizado como fonte deste documento os arquivos disponibilizados nas Avaliações Objetivas 1 e 2 (Aulas 11 e 22). Todo material está disponível no repositório do GitHub.

1. Fundamentação Lógica e Domínio de Fluxo

A linearidade é a exceção no software profissional. Sistemas reais devem reagir a dados dinâmicos e validar regras complexas.

1.1. Estruturas de Decisão (if, elif, else)

As estruturas de decisão formam a **Lógica de Negócio (Business Logic)**, que é o que torna o software valioso para as empresas. No cenário de uma **Fintech (Caixa Eletrônico)**, a lógica define se um patrimônio será protegido ou se uma transação será negada.

- Sintaxe e Indentação (PEP 8):** Python substitui as chaves `{ }` pela indentação obrigatória. Isso força a organização visual e evita o "Código Macarrônico".
- Avaliação de Curto-Circuito:** O interpretador Python é eficiente. Em um bloco `if/elif`, se a primeira condição for `True`, as demais não são avaliadas, o que **economiza processamento**.

1.2. Estruturas de Repetição (while vs. for)

Estrutura	Foco Principal	Risco / Recomendação
<code>while</code>	Estado/Condição	Risco de Loop Infinito se a variável de controle não for alterada. Ideal para menus que rodam até um comando "Sair".
<code>for</code>	Dado/Iterador	Mais seguro e performático. Evita o erro clássico de "índice fora dos limites" (<i>Out of Bounds</i>). Escolha padrão para 90% das iterações profissionais.

1.3. Controle de Execução (break e continue)

- break:** Interrompe o laço imediatamente. Em um **monitoramento de sensores industriais**, se um sensor detecta fumaça, não podemos esperar o fim do ciclo; o `break` permite o desvio imediato para rotinas de emergência.
- continue:** Salta para a próxima iteração. Útil para ignorar dados (ex: pular funcionários em licença não remunerada durante o cálculo da folha).

1.4. Boas Práticas de Escrita

- Nomes Semânticos:** Use `valor_saque` em vez de `vs`. O código deve ser autoexplicativo.
- Constantes:** Valores que não mudam (ex: `LIMITE_DIARIO = 500`) devem estar em **MAIÚSCULO**.
- f-strings:** Utilize `f"Saldo: {valor:.2f}"` para logs claros e formatados.

2. Modularização e Persistência de Arquivos

2.1. Arquitetura Modular

Projetos profissionais utilizam a **Separação de Responsabilidades (Separation of Concerns)**. Dividimos o código em múltiplos arquivos `.py` para que lógicas financeiras não se misturem com interfaces, facilitando o *debugging* e o trabalho em equipe.

2.2. A Variável Especial `__name__`

O bloco `if __name__ == '__main__':` garante que o código de teste só execute se o script for chamado diretamente. Se o arquivo for importado, a variável `__name__` assume o **nome do arquivo**, e o bloco de execução principal é ignorado, evitando execuções indevidas.

2.3. Persistência Básica em Texto

Dados na RAM são voláteis. Utilizamos `open()` com o gerenciador de contexto `with` para garantir que o arquivo seja fechado automaticamente, prevenindo corrupção de dados.

- **'r' (read):** Leitura. Gera `FileNotFoundError` se o arquivo não existir.
 - **'w' (write):** Escrita. Sobrescreve tudo o que houver no arquivo.
 - **'a' (append):** Anexo. Adiciona dados ao final sem apagar os anteriores.
-

3. Programação Orientada a Objetos (POO)

3.1. Classes e Objetos

A POO aproxima o código da compreensão humana.

- **Classe (O Molde):** A planta baixa que define atributos e métodos.
- **Objeto (A Peça):** A materialização da classe na memória com dados reais.

3.2. O Método Construtor (`__init__`) e Atributos

O `__init__` padroniza a criação de entidades. O `self` é a referência que guarda o estado dentro de cada objeto específico.

3.3. Métodos e Encapsulamento

Métodos são as ações (verbos). O encapsulamento protege a lógica: alterar o saldo de um cliente manualmente fora da classe é considerado um **falha de design**. O correto é invocar o método `depositar()`, que contém as validações de segurança.

3.4. Herança e Especialização

Representa a relação **"É UM"**. Permite o reaproveitamento de código e a especialização de comportamentos.

Exemplo de Herança e `super()`

```
class Veiculo:
```

```
    def __init__(self, marca, ano):
        self.marca = marca
        self.ano = ano
```

```
class Caminhao(Veiculo):
```

```
    def __init__(self, marca, ano, capacidade_carga):
        super().__init__(marca, ano) # Reaproveita o construtor da classe mãe
        self.capacity = capacidade_carga
```

4. Banco de Dados Relacional e Padrão DAO

4.1. Mapeamento Objeto-Relacional (ORM) e DDL

O DDL (*Data Definition Language*) cria o "lar permanente" dos dados.

-- DDL para criação robusta da tabela

```
CREATE TABLE IF NOT EXISTS tb_produto (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL,
```

```
preco REAL NOT NULL
);
```

4.2. Conectividade Robusta

O módulo `conexao.py` deve usar `try/except` para tratar falhas de I/O. Confiar no "caminho feliz" (sem erros) é um erro de design grave que causa *crashes* e perda de dados.

4.3. Operações CRUD Seguras

O descuido com o SQL pode gerar um **"Code Bomb"**: um `UPDATE` ou `DELETE` sem a cláusula `WHERE` pode zerar o estoque de uma empresa ou apagar todos os clientes em milissegundos.

- **Estratégia Defensiva:** Sempre realize um `SELECT` para validar o ID antes de alterar ou excluir.
- **A Regra da Vírgula (Tuple Comma):** Ao passar um único parâmetro no `execute()`, você **deve** usar a vírgula: `(id_produto,)`. Sem ela, o Python não reconhece a Tupla, causando erros de *bindings*.
- **Soft Delete:** No mercado corporativo, raramente excluimos dados fisicamente. Usamos o `UPDATE` para mudar um status para 'Inativo'.

Exemplo de UPDATE Seguro com Placeholders e Tupla

```
cursor.execute("UPDATE produtos SET preco = ? WHERE id = ?", (novo_preco, id_produto))
conexao.commit() # Vital para gravar no disco
```

5. Testes Unitários e Rastreabilidade

5.1. O Padrão DAO (Data Access Object)

O DAO isola o SQL do Python. Ele é o intermediário que permite testar a persistência sem poluir a lógica de negócio.

5.2. Framework `unittest` e Ciclo de Vida

Adotamos a filosofia **"Back-End First"**: não construímos interfaces sem provar que os dados são salvos e rastreáveis.

```
import unittest
import sqlite3
```

```
class TestDAO(unittest.TestCase):
    def setUp(self):
        # Uso de :memory: para testes isolados e rápidos na RAM
        self.conn = sqlite3.connect(':memory:')
        self.conn.execute("CREATE TABLE produtos (id INTEGER PRIMARY KEY, nome TEXT)")

    def test_persistencia(self):
        # Action, Query e Assert
        cursor = self.conn.cursor()
        cursor.execute("INSERT INTO produtos (nome) VALUES (?)", ("Teste",))
        self.assertIsNotNone(cursor.lastrowid) # Rastreabilidade do ID
```

5.3. Garantia de Qualidade

- **Rastreabilidade:** Identificar onde o dado se perde entre a RAM e o Disco.
 - **Defesa de Dados:** Usamos `assertRaises` para garantir que o sistema rejeite dados inválidos (como preços negativos) antes que cheguem ao banco.
-

6. Introdução ao Desenvolvimento Web (Flask)

6.1. Transição Back-End para Interface

O Flask é a camada visual que consome a inteligência estável do banco de dados. A interface é apenas o "rosto" do sistema; a segurança reside no Back-End.

6.2. Arquitetura Web

- **Rotas:** Endereços de acesso.
- **Métodos (POST):** Para envio seguro de informações.
- **Jinja2:** Renderização de HTML dinâmico com dados do Python.

6.3. Padrão PRG (Post-Redirect-Get)

Sistemas profissionais não permitem que o usuário reenvie um formulário ao atualizar a página (o que poderia duplicar uma compra). O padrão **PRG** redireciona o usuário para uma rota de sucesso, garantindo a resiliência da plataforma e uma experiência de usuário superior.